

Self-Billing Counter System

Platform: EMU8086 (CISC Assembly)

Introduction

This project replicates a retail billing system using low-level CISC (Complex Instruction Set Computing) assembly language on the EMU8086 platform. Every functionality, including storing member data and calculating bills, is implemented simply by utilizing registers, memory arrays, macros, and procedures.

Users

- **Admin** - manages products, views all purchase history
- **Member** - registers with phone number, earns loyalty points
- **Non-Member** - shops without registration, no points

Features

- **Member Register** > Allows to register new members by entering their name, phone. Member information is stored in arrays and assigns the phone number as user ID.
- **Member Log in** > Members can log in using their phone number and do shopping.
- **Non-member shopping** > Even Non-members can do shopping without any log in credentials.
- **Admin log in** > Admin log in through passcode.
- **Admin Control** > Can make changes like product stock update, adding new product with price (multiple of 5).
- **Product listing** > Shows all the products with prices and their availability.
- **Product scanning(I/O)** > Allows to add product by (scanning) entering their 3 digit uniques id push them in cart (stack).
- **Cart list generation** > Generates an cart list
- **Payment system (cash change, point)** > Allows to pay through cash note (multiple of 100) and get exchange amount (multiple of 50). Only Member gets points (purchase amount//100). Points can be used at the next Purchase due to payment.
- **Bill_print** > Prints bill after payment.
- **Member purchase history** > Members can watch their purchase history.
- **Admin all-over history** > Admin can watch all the purchase history of member and non-member.
- **Error Handle** > Handles all invalid inputs/data issues.

Few code overview

- **Arrays:**

All data is stored as flat parallel arrays, one per field, and indexed in the same way. This is the assembly equivalent to struct arrays.

```
# Enter Code Here

; Product arrays - 5 pre-filled defaults
PROD_CNT DW 5
PROD_ID DW 101,102,103,104,105,0,0,0,0,0
PROD_NM DB "Rice  ", "Oil   ", "Sugar ", "Salt  ", "Flour  "
PROD_PR DW 100,150,100,50,200,0,0,0,0,0
PROD_ST DW 50,30,40,100,20,0,0,0,0,0
```

- **Stack:**

The shopping cart is constructed as a stack, with two parallel arrays and a counter as the top pointer.

```
# Enter Code Here

;
CART_CNT DW 0           ; stack top
CART_PI DB 0,0,0,0,0,0,0,0,0,0 ; product index (byte per slot)
CART_QY DW 0,0,0,0,0,0,0,0,0,0 ; quantity (word per slot)
```

```
# Enter Code Here

;
MOV SI, CART_CNT
MOV CART_PI[SI], BL ; store product index
MOV AX, CART_CNT
ADD AX, AX
MOV SI, AX
MOV CART_QY[SI], CX ; store quantity
INC CART_CNT        ; advance stack top
```

PUSH (adding an item to cart)

POP (reading during bill)

- **Macros (MACRO):**

Macros are reusable in-line code functions. They grow at the moment when using **CALL/RET** overhead.

- **Key press:**

ONE_KEY - Single Key press (press any key)

Enter Code Here

;

ONE_KEY MACRO

MOV AH, 01h

INT 21h ; AL = key pressed

MOV BL, AL ; save immediately before AH is corrupted

ENDM

○ **Fixed-Digit Input Macros (No Enter/Key Needed):**

The design decision is to make all numeric inputs fixed-digit and auto-progress, allowing users to enter exactly N digits and the application to begin instantly. No key is necessary to be pressed.

Enter Code Here

; THREE_DIG_IN -- read exactly 3 digit keys -> DEST (word)

; Used for product ID scan (100-999) and 000 to finish cart.

THREE_DIG_IN MACRO DEST

MOV DEST, 0

MOV AH, 01h ; hundreds digit

INT 21h

MOV BL, AL ; SAVE immediately — AH corrupted by DOS after INT

SUB BL, 30h ; convert ASCII to digit value

MOV CL, BL

MOV CH, 0

MOV AX, 100

MUL CX

ADD DEST, AX

MOV AH, 01h ; tens digit

INT 21h

MOV BL, AL

SUB BL, 30h

MOV CL, BL

MOV CH, 0

MOV AX, 10

MUL CX

ADD DEST, AX

MOV AH, 01h ; ones digit

INT 21h

MOV BL, AL

SUB BL, 30h

MOV CL, BL

MOV CH, 0

MOV AX, 1

MUL CX

ADD DEST, AX

ENDM

- **Procedures (PROC):**

Procedures are invoked with **CALL** and returned with **RET**. They save and restore all the registers they utilize (PUSH/POP), therefore callers are unaffected.

- **PROC_FIND_PROD - Search Product by ID:**

Comparing 3 digit input with stored data and finding matched data.

```
# Enter Code Here
;
PROC_FIND_PROD PROC NEAR
    PUSH BX
    PUSH CX
    PUSH SI
    MOV CX, PROD_CNT    ; loop count = active products
    MOV BX, 0           ; BX = current index
FNDPR_LP:
    MOV AX, BX
    ADD AX, AX          ; word offset = index * 2
    MOV SI, AX
    MOV AX, PROD_ID[SI] ; load product ID at this index
    MOV DX, AX          ; save to DX before CMP
    CMP DX, TMP_NUM     ; match with search target?
    JE FNDPR_OK
    INC BX
    LOOP FNDPR_LP
FNDPR_FAIL:
    MOV AX, 255         ; not found
    ...
    RET
FNDPR_OK:
    MOV AX, BX          ; return found index
    ...
    RET
PROC_FIND_PROD ENDP
```

- **Product Scanning and Cart Feature:**

Add products by entering their 3 digit unique id and push them in stack.

```
# Enter Code Here
;
SCAN_LP:
    PRINT_STR S_SCAN
    THREE_DIG_IN TMP_NUM ; user types 3 digits -> stored in TMP_NUM
    NEWLINE
    MOV AX, TMP_NUM
    MOV BX, AX
    CMP BX, 0           ; 000 = done scanning
    JE SCAN_DONE

    CALL PROC_FIND_PROD ; search for this ID -> AX = index or 255
```

```
MOV BX, AX
CMP BX, 255
JNE SCAN_FOUND
PRINT_STR S_NF
JMP SCAN_LP

SCAN_FOUND:
; check stock, read qty, push to cart stack
MOV SI, CART_CNT
MOV CART_PI[SI], BL ; push product index (byte)
MOV AX, CART_CNT
ADD AX, AX
MOV SI, AX
MOV CART_QY[SI], CX ; push quantity (word)
INC CART_CNT
```

Limitations:

The system has some limitations. Some limitations are set intentionally for feature development.

- Data is not saved to disk - all arrays reset when the program exits.
- Cash input is capped at 5 digits (max 32700 due to 16-bit word limit).
- Member phones must be exactly 10 digits.
- Price must be typed as 5 digits (e.g. 00100 for 100 cash).
- Admin price must be a multiple of 5.
- Member payment cash amount must be a multiple of 100.

Conclusion:

This project demonstrates that a fully functional interactive commercial application can be built entirely in 8086 assembly. Parallel array data structures, stack-based cart management, fixed-digit input macros (no key), linear search procedures, state of session flag values, as well as the critical rule of saving register values immediately after **INT 21h** before any comparison or calculation are among the used key concepts.